Reproducible Builds: Increasing the Integrity of Software Supply Chains

Chris Lamb, Reproducible Builds

Stefano Zacchiroli, Université de Paris and Inria

// Although it is possible to increase confidence in free and open source software by reviewing its source code, trusting code is not the same as trusting its executable counterparts. This article examines reproducible builds, an approach that can determine whether generated binaries correspond to the original source code. //

©SHUTTERSTOCK.COM/ALEXDNDZ

You can't trust code that you did not totally create yourself. [...] No amount of source-level verification or scrutiny will protect you from using untrusted code.

—Ken Thompson (1984)

Digital Object Identifier 10.1109/MS.2021.3073045
Date of current version: 14 February 2022

HOW CAN WE be sure that our software is doing only what it is supposed to do? This was the key takeaway from Ken Thompson's 1984 Turing Lecture, "Reflections on Trusting Trust."[1] But with people today executing far more software than they compile, the number of users who "totally create" software they run has dropped dramatically since then.

Let us narrow the issue to free and open source software (FOSS), where all source code is freely available. Hypothetically, users can examine the source of all the software they wish to use to confirm that it does not contain spyware or backdoors; indeed, one of the original promises of FOSS was that distributed peer review[2] would result in enhanced end-user security. However, while users can inspect source code for malicious flaws, almost all software is now distributed as prebuilt binaries. This permits nefarious actors to compromise end-user systems by modifying ostensibly secure code during its compilation or distribution.

For example, a Linux distribution might compile "safe" software on compromised servers and unwittingly spread malicious executables onto countless systems. Other vectors include engineers being explicitly coerced into incorporating vulnerabilities as well as the covert compromise of developers' computers (remotely or through "evil maid" attacks[3]) so that they unwittingly distribute tainted binaries via app stores and other channels.

Software supply-chain attacks are no longer hypothetical scenarios. In December 2020, news broke that attackers subverted the SolarWinds Orion software to inject malicious code into executables at build time, resulting in a severe data breach across several U.S. government branches.[4] One-hundred and seventy-four similar attacks have been detailed in the literature too.[5] Due to their potential impact, software supply chains have become a high-value target in recent years, and this trend appears to be accelerating.

Practical and scalable solutions to these attacks are therefore urgently

needed, and an approach known as *reproducible builds* (*R-B*) is one such countermeasure. However, it is only applicable if the software sources are widely available—although malware can be detected directly in binaries[6,7], doing so is less efficient than auditing source code.

The key idea behind the R-B approach is that, if we can guarantee that building a given source tree always generates bit-for-bit identical results, we can establish trust in these artifacts by comparing outputs acquired from multiple, independent builders. In this article, we present the R-B approach from the perspective of software professionals. We show how software users can benefit from the increased trust in executables they run as well as how developers and build engineers can help make software reproducible. We also describe the quality assurance (QA) tools available to improve build reproducibility, highlighting how they further mutually beneficial goals such as reducing build and test "flakiness."[8,9] This article is informed, in large part, by the experience of the Reproducible Builds project (reproducible-builds.org), a nonprofit initiative that popularized the R-B approach.

Overview

The core element of the R-B model is the following property.

Definition 1

The build process of a software product is reproducible if, after designating a specific version of its source code and all of its build dependencies, every build produces bit-for-bit identical artifacts, no matter the environment in which the build is performed. In other words, once we reach an agreement on the exact software version(s) we wish to build, anyone who builds that software should always generate precisely the same artifacts. ■

Figure 1 shows how users can leverage this property to establish trust in FOSS executables. Software development happens upstream as usual (e.g., on platforms such as GitHub and GitLab), and from there, source code reaches downstream vendors such as Linux distributions and app stores. These vendors then build binaries from these sources, which are subsequently distributed to end users. Note that neither the distribution nor the build process are completely trusted in this scenario, reflecting the hostile environment of the real world.

When software builds reproducibly, however, we can still establish trust in these executables. This is because users can corroborate whether their newly downloaded binaries are identical to those that others have built themselves.

How this works is as follows: At the top of the supply chain, trust in a specific version of a piece of software is established through auditing the source code or, more likely, by implicitly trusting its developers (e.g., trusting version 5.11.5 of the Linux

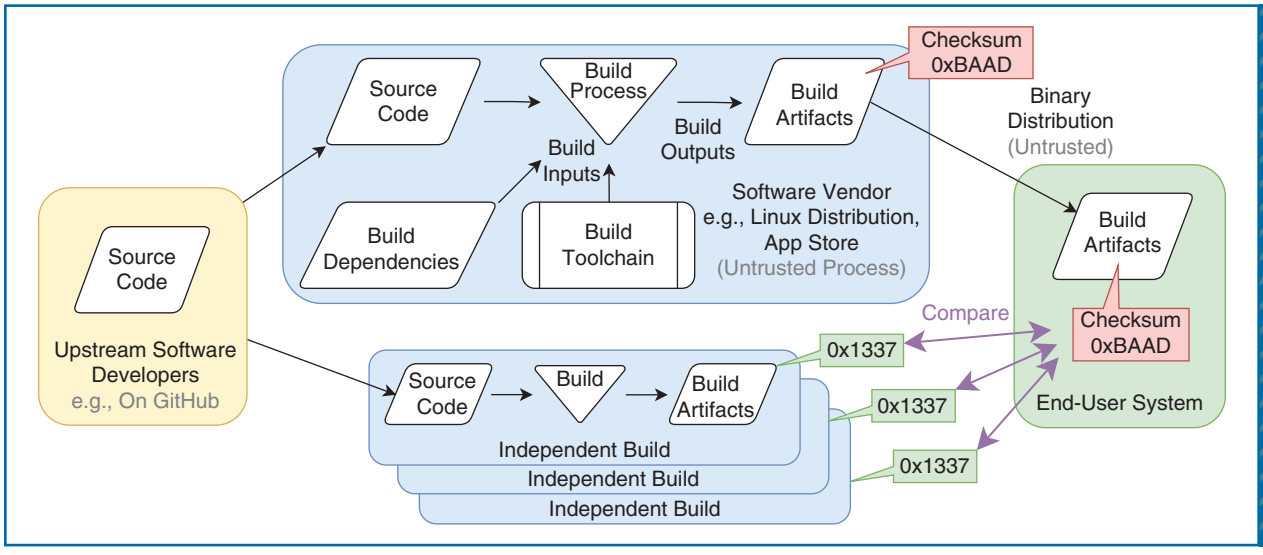

FIGURE 1. The R-B approach to increasing trust in executables built by untrusted third parties. The end-user should reject the binary artifact from their software vendor, as its checksum (`0xBAAD`) does not match the one built by multiple independent third parties (`0x1337`).

kernel as it is signed by Linus Torvalds). Later, but before executing any binaries they have downloaded, users can compare the checksums of these files with the expected values for that specific version, crucially aborting on any mismatch.

These expected checksums could originate from a limited set of trusted parties who publish statements that building some specific source code release results in a particular set of executables. However, another alternative is distributed consensus, where a loose-knit community of semitrusted builders independently announce their checksums. Normally, the participants in this scheme report identical checksums for a given source code release, but in case of a discrepancy (i.e., if some builders have been compromised), the checksum reported by less than 50% of the builders may be the one to trust. Under this verification scheme, the takeover of at least 50% of the builder community would be required to coerce users into running malicious binaries.

> Software supply chains have become a high-value target in recent years, and this trend appears to be accelerating.

Reproducibility in the Small

How hard is it to ensure that independent builds always result in bit-for-bit identical executables? First, let us consider the software vendor in Figure 1. Each build takes as its input the source to be built, all of its build-time dependencies, and the entire build toolchain, including the compiler, linker, and build system. The build produces a set of artifacts (executables, data, documentation, and so on) as its output. Any change in these inputs may legitimately affect its output.

However, even once all the inputs have been controlled for, the build may still be unreproducible—that is, producing different artifacts when the build is repeated. This results from two main classes of problem: uncontrolled build inputs and build nondeterminism.

Uncontrolled build inputs occur when toolchains allow the build process to be affected by the surrounding environment. Common examples include system time, environment variables, and the arbitrary build location on the filesystem. Uncontrolled inputs can be seen as analogous to breaking encapsulation in software design; a tight coupling between a high-level process and the low-level implementation details.

Build nondeterminism occurs when aspects of the build behave nondeterministically, and these "random" behaviors are encoded in the final artifacts. For example, if the output is derived in any way from the state of a pseudorandom number generator or the arbitrary order of process scheduling.

To address uncontrolled build inputs, it is tempting to "jail" builds into sanitized environments that always present a canonical interface to the underlying build system. Indeed, this was the approach taken by early projects such as Bitcoin and Tor (rbm.torproject.org). However, jails result in slower build times and impose technical and social restrictions on developers who may be accustomed to choosing their tooling. Most jails cannot address nondeterminism issues either.

The ultimate and preferred solution is to ensure that any code run during the build only depends on the legitimate build inputs (the source being built, the build dependencies, and the toolchain) and that any nondeterministic behavior does not affect the resulting artifacts. We now review some individual causes of unreproducible builds and show how to address them.

Build Timestamps

Timestamps are, by far, the biggest source of unreproducibility. It is a common practice to explicitly embed dates into binaries via C's __DATE__ macro (see Listing 1), but many tools

```
void usage() {
    fprintf (stderr,
            "foo-utils version "
            "3.141 (built %s)\n",
             __DATE__);
}
```

LISTING 1. The __DATE__ C preprocessor macro "expands to a string constant that describes the date on which the preprocessor is being run."

record dates into build artifacts as well. For example, **help2man** generates Unix manual pages directly from the output of **--help**, and in its default configuration, it embeds the current date into generated files. As this value changes from day to day, this results in an unreproducible build. TEX's **\date** macro also embeds the current date, with similar implications for generated documentation.

The value of these timestamps is extremely limited, particularly as they do not convey which version of the software was actually built; after all, older software can always be built later. Embedded timestamps should therefore be avoided entirely, but for cases where that is not possible (e.g., in file formats that mandate their presence), the Reproducible Builds project proposed the **SOURCE_DATE_EPOCH** environment variable as a way to communicate an acceptable timestamp to build systems.[10] This typically represents the last modification time of the source tree as extracted from the software's changelog file.

Build Paths

The file system path where the build took place is often embedded in generated binaries too, usually via the **__FILE__** preprocessor macro (see **Listing 2**) or by assertion statements that reference their corresponding line of code. Other sources of build paths include logging messages, locations of detached debug symbols, **RPATH** entries in ELF binaries, and many other instances that are intended, ironically, to assist the software development process.

To help address this issue, the Reproducible Builds project worked with the GNU GCC compiler project developers to introduce the **-ffile-prefix-map** and **-fdebug-prefix-map** options, which support embedding relative (rather than absolute) paths.

```
fprintf (stderr,
        "DEBUG: boop (%s:%s\n",
        __FILE__, __LINE__);
```

LISTING 2. The __FILE__ C preprocessor macro "expands to the name of the current input file." This results in nonreproducibility when the program is built from different directories, e.g., **/home/lamby/tmp** versus **/home/zack/tmp**.

File System Ordering

Contrary to the output of **ls (1)**, the Portable Operating System Interface Unix standard does not specify an ordering for the results returned by the underlying **readdir(3)** system call. As a result, the directories accessed in naive "**readdir** order" may be processed in a nondeterministic manner. If this arbitrary ordering influences any build artifacts, the build will not be reproducible.

For example, the build system of the PikePDF library located its own source files using Python's **glob** routine. But as **glob**'s result value inherits the nondeterminism of **readdir(3)**, PikePDF's source files were linked in an arbitrary order.

This is a particularly pernicious problem as some file system implementations return different orderings more often than others. To avoid these issues, build systems should impose a deterministic order on any directory iteration encoded in its artifacts, e.g., via an explicit **sort()**.

Archive Metadata

.zip and **.tar** archives store timestamps and user ownership information in addition to the files themselves. However, if this metadata is inherited from the surrounding build environment, it will not be replicated when building elsewhere. For example, if a **.tar** archive stores files as belonging to the build user (e.g., **lamby**), another user (e.g., **zack**) building the same software will obtain a different result.

This can be avoided by instructing tools to ignore on-disk values in favor of metadata chosen by the build system (e.g., using **tar (1)** with **--owner=0** and **--clamp–mtime=T**) or by normalizing the metadata before archiving begins (e.g., by using **touch(1)** with **SOURCE_DATE_EPOCH** as a reference timestamp).

Randomness

Even when the entire environment is controlled for, many builds remain inherently nondeterminstic. For example, builds that iterate over hash tables (such as Perl's hash or the **dict** type in Python <3.7) exhibit arbitrary behavior as their respective elements are returned in an undefined order; the code in Listing 3, for example, may print any combination of **abc**, **bac**, **bca**, and so forth. This affects the reproducibility if these results form any part of the build's artifacts.

Parallelism (such as via processes or threads) can also prevent reproducibility if the arbitrary completion order is encoded into build results too. Similar to file system ordering, these issues can be resolved by imposing determinism in key locations, seeding any sources of randomness to fixed values or sorting the results of hash iterations and parallelized tasks before generating output.

```
my %h = (a => 1, b => 2, c => 3);
foreach my $k (keys %h) {
    print "$k\n";
}
```

LISTING 3. Perl's hash type does not define an ordering of its keys, so a call to **sort** should be inserted before **keys %h** to make it deterministic.

Uninitialized Memory

Many data structures have undefined areas that do not affect their operation. The file allocation table (FAT) filesystem, for example, contains unused regions that may be filled with arbitrary data. In addition, modern CPU architectures perform more efficiently when data are naturally aligned, and the padding added to ensure alignment can result in similarly undefined areas. These regions containing random data affect reproducibility when stored in build results.

One solution is to explicitly zero-out memory regions that may persist in artifacts. For example, Listing 4 shows a patch for GNU mtools, which ensures that generated FAT directory entries do not embed uninitialized memory.

```
--- a/direntry.c
+++ b/direntry.c
@@ -24,6 + 24,7 @@
  void initializeDirentry(
      direntry_t *entry, Stream_t *Dir) {
+     memset(entry, 0, sizeof(direntry_t));
      entry->entry = -1;
      entry->Dir = Dir;
```

LISTING 4. A patch for GNU mtools ensuring that a direntry_t struct does not contain uninitialized memory.

Reproducibility in the Large

Now that we know how to address some individual reproducibility issues, we turn our attention to the problems that arise when making large software collections reproducible. The Reproducible Builds project started in 2014 with the aim of making the Debian operating system (www.debian.org) completely reproducible. This is a formidable goal as not only is Debian an extremely mature Linux distribution, it is one of the largest curated collections of FOSS software in general.

> Build nondeterminism occurs when aspects of the build behave nondeterministically and these "random" behaviors are encoded in the final artifacts.

Seven years later, more than 95% of the 30,000-plus packages in Debian's development branch can now be built reproducibly, and as the Linux distribution with the largest total number of reproducible packages, it serves as an extremely relevant case study. The evolution of this effort can be found at wiki.debian.org/ReproducibleBuilds.

Adversarial Rebuilding

Given its scale, the Debian developers realized that they would need a programmatic way to test for reproducibility. To this end, they developed a continuous integration (CI)[11] system that builds each package in the Debian archive twice in a row, using two independent build environments that are deliberately configured to differ as much as possible. For instance, the clock on the second build is set 18 months in the future, and the hostname, language, system kernel, and so on are all varied so that if any environmental differences are used as a build input, the two builds will differ as a result. The large number of variations applied (30 plus) can validate build reproducibility to a high degree of accuracy.

To identify any reliance on nondeterministic file system ordering, the R-B project also developed a Filesytem in Userspace (FUSE)-based[12] virtual file system called *disorderfs* (salsa.debian.org/reproducible-builds/disorderfs), which can provide a view of a file system with configurable orderings. The R-B CI system reverses the file system ordering between the builds, revealing any dependency on nondeterministic file system ordering.

Recording Build Information

As per Definition 1, a reproducible build must always use the same original source, toolchain and build dependencies, and to ensure that these inputs can be replicated correctly, Debian devised the .buildinfo file format. Once a Debian package is built, the precise source version and the versions of all its build dependencies are recorded in a .buildinfo file. This file also contains checksums of any generated .deb artifacts—that is, the Debian binary package format. (An example file may be found in Listing 5.)

.buildinfo files are a crucial building block for any process wishing to validate reproducibility. A .buildinfo is produced during an initial build and is then used to reconstruct a second build environment. The build is repeated within this second environment, and the checksums from this latter build are compared with the ones in the

original .buildinfo. If they do not match, the build is unreproducible or a build host has been tampered with.

Users can employ .buildinfo files to implement the consensus-driven approach outlined previously, verifying downloaded packages by comparing them against the checksums in the .buildinfo files distributed by Debian and other builders. In this scenario, .buildinfo files are cryptographically signed to represent a build attestation, e.g., "I, Alice, given source X and environment Y, have built a package with checksum K." Bob would verify that Alice really made this claim and then compare Alice's K against his downloaded file, potentially trusting Alice's K over any divergent (and likely malicious) claim from Eve.

Debian currently hosts more than 20 million .buildinfo files in a number of experimental services. However, centralized distribution schemes inherit many of the issues of the SSL certificate authority ecosystem, particularly in representing an obvious target to attack.[13] Decentralized alternatives remain a future challenge at this point as does a practical consensus mechanism to determine the valid checksum for any given package.

Root-Cause Analysis

As outlined previously, it is trivial to detect mismatches between builds simply by comparing the checksums of their artifacts. However, it can be extremely difficult to understand the root cause of this difference.

Therefore, the R-B project developed diffoscope (diffoscope.org), a visual "diff" tool that recursively unpacks a large number of archive formats and translates tens of binary formats into human-readable forms. As a result, it can display the meaningful, code-level differences between, for example, two compiled Java .class files, even if they were contained in .tar in a .xz (in a .deb, .iso, and so forth).

In most cases, diffoscope indicates which category of fix is required to make a build reproducible. For instance, when programs embed build dates into their binaries, diffoscope clearly highlights these date-based variations, and the surrounding context tends to assist in identifying which part of the original source code to fix.

An example diffoscope output is shown in Figure 2, where two versions of the dolfinx-doc package differ. Here, diffoscope indicates that the difference is in CMakeLists.txt, a generated file which contains the same entries with a different ordering between the two builds. This would appear to be a file system ordering issue, solved by the addition of an explicit sort. However, this may be a problem affecting all software that uses CMake, so the issue may be better addressed there; alas, diffoscope cannot entirely replace a software engineer's judgment.

Quality Assurance

Adopting a methodical approach to verify the reproducibility of builds can be highly complementary to QA efforts. This is because the problems that affect build reproducibility are often symptoms of larger, systemic issues.

To begin with, systematic reproducibility testing implies systematic build testing, so it will easily identify software that fails to build under any circumstances. Other software will fail to build only in the extreme environments designed to test for reproducibility but will become more robust as a result of behaving well there. For example, some software will fail to build in the future due to hardcoded SSL certificates with expiration dates; these are detected due to the build environment's artificial future clock. Others fail to build in rarely used time zones due to incorrect assumptions about time offsets; the test suite for the Ruby Timecop library failed in this way (bugs.debian.org/795663).

Due to these serendipitous quality improvements, addressing reproducibility issues improves the correctness and robustness of the Debian distribution as a whole. However, even less-critical problems can be identified through reproducibility testing. For example, an issue in the Doxygen documentation generator (bugs.debian.org/970431) led to broken hyperlinks that linked to their

```
Source: black
Version: 20.8b1-1
Checksums-Sha1:
 9915459ae7a1a5c3efb984d7e5472f7976e996b1 2584 black_20.8b1-1.dsc
 14bfd3011b795f85edbc8cc4dc034a91cfaa9bcd 111096 black_20.8b1-1_all.deb
 69c3d4ae7115c51e7b00befe8b4afd5963601d66 285684 python-black-doc_20.8b1-1_all.deb
Checksums-Sha256: […]
Build-Architecture: amd64
Installed-Build-Depends: autoconf (= 2.69—11.1), automake (= 1:1.16.2-4), […],
 gcc (= 4:10.2.0-1), […], python3 (= 3.8.2-3), […]
 xz-utils (= 5.2.4-1+b1), zlib1g (= 1:1.2.11.dfsg-2)
```

LISTING 5. An example .buildinfo file, recording both the environment and results of building Debian's black package. (Excerpt: see buildinfo.debian.net/sources/black/20.8b1-1 for the full version.)

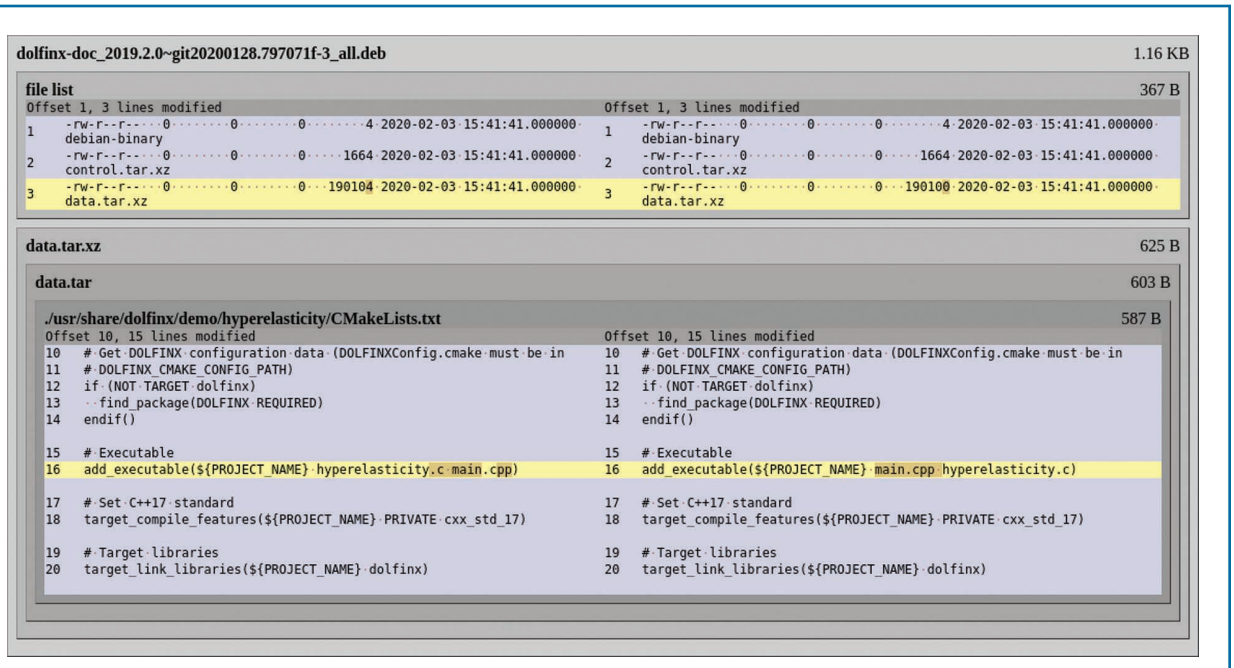

FIGURE 2. A diffoscope recursively unpacks archives of many kinds and transforms various binary formats into more human-readable forms for comparison.

build-time location (e.g., /tmp/build/foo/usage.html) instead of their runtime one (/usr/share/doc/foo/usage.html). This was trivial to identify with diffoscope, and fixing it corrected broken documentation for hundreds of end users.

Reproducibility testing can even flag spurious content within documentation. For instance, manual packages generated by executing an underlying program can fail in several ways, often printing error messages that are mistakenly shipped as the package's documentation (e.g., bugs.debian.org/972635). These bugs are difficult to detect if the failure does not occur on a developer's own machine, but they can be easily spotted while testing for reproducibility as the error messages are deliberately designed to appear in different languages.

Even security issues can be discovered while testing for reproducibility. In one example, the GBrowse biological genome annotation viewer failed to build reproducibly (bugs.debian.org/833885), and diffoscope identified a configuration file that contained a different OpenIDConsumerSecret value between builds (see Listing 6). Although this secret was being securely generated, it was being created at build time, so the same value was distributed to all users of the package; the fix was supposed to generate the secret at installation time so that each deployment possessed its own unique key. The mechanics of reproducibility testing suggest that this issue would not have been readily discovered another way.

Community Engagement

Although the causes of build unreproducibility often reside within the source code of individual projects, it is far more effective to detect issues via centralized testing in distributions such as Debian due to the uniform build interfaces these large collections provide. Nevertheless, the social norms of the FOSS community dictate that fixes should be integrated upstream, instead of remaining in distribution-specific patch sets.

To this end, the Reproducible Builds project has contributed to

```
{
  'cgibin' => '/usr/lib/cgi-bin/gbrowse',
  'conf' => '/etc/gbrowse',
  'databases' => '/var/lib/gbrowse/databases',
  'htdocs' => '/usr/share/gbrowse/htdocs',
  'OpenIDConsumerSecret' =>
     '639098210478536',
  'tmp' => '/var/cache/gbrowse'
},
```

LISTING 6. An example ConfigData.pm. As it was created at build time, all users shared the same OpenIDConsumerSecret.

hundreds of individual FOSS projects, in addition to working with key toolchains such as GCC, Rust, OCaml, and so on. This community-oriented approach ensures that as many users as possible can benefit from the specific advantages of R-B as well as from the software quality improvements achieved while pursuing that goal.

The R-B Ecosystem

Taking Debian to its current state required more than seven years of cross-community work, which was spearheaded by the Reproducible Builds project (reproducible-builds.org), a nonprofit organization that aims to increase the integrity of software supply chains by advocating for and implementing the approach outlined in this article.

Although originating in Debian around 2014, many other FOSS projects have joined the initiative, such as Arch Linux, coreboot, F-Droid, Fedora, FreeBSD, Guix, NixOS, openSUSE, and Qubes. One milestone of this joint effort is Tails (tails.boum.org/), the operating system used by Edward Snowden to securely communicate the National Security Agency's global surveillance activities in 2013[14]. Tails began releasing reproducible .iso images in 2017 to improve end-user verifiability and security.

The Reproducible Builds project has also developed several tools (reproducible-builds.org/tools) that facilitate the various QA processes related to reproducibility. Some of these, such as diffoscope and disorderfs, have been highlighted in this article.

Increasing the security of open source software is clearly a worthwhile goal, and software professionals and organizations can always provide assistance. This is accomplished not only by addressing any uncontrolled build inputs and sources of nondeterminism in the software they maintain, but by working with the Reproducible Builds project itself in terms of code, donations, and other traditional forms of community contribution.

In this article, we outlined what it means for software to build reproducibly and how that property can be leveraged by end users to establish trust in open source executables, even when they are built by untrusted third parties. We also surveyed several causes of unreproducibility and located their causes in build systems and in similar logic. We also described some of the QA processes and tools that can be used to make large open source software collections reproducible. Using this model, the Debian operating system has achieved 95% reproducibility in more than 30,000-plus packages.

> Uncontrolled build inputs occur when toolchains allow the build process to be affected by the surrounding environment.

Additional work is still needed to address the software that is not yet reproducible. In the case of Debian, there are no insurmountable obstacles preventing the project from reaching 100%. The remaining 5% need only fixes similar in kind to those already discussed. However, this has not yet been achieved, partly because time and effort are not inexhaustible (or fungible) resources in volunteer communities, but also due to regressions in the previously reproducible packages. Improved awareness and prioritization of reproducibility among software developers would reduce the incidence of such events.

Other challenges remain for the R-B ecosystem too. Cryptographically signed artifacts are becoming more common, which cannot be made reproducible without distributing signing keys to builders. One solution is to adopt detached signatures, but the addition of parallel distribution channels for these (unreproducible) files would require extensive changes to existing software distribution channels.

The verification of open source software for mobile devices also remains problematic. With the notable exception of F-Droid, not only are the build processes of the major app stores unreproducible (or not even FOSS), the checksums of artifacts are hidden from end users, rendering any distributed validation scheme impossible. Significant usability and transparency improvements are needed to make meaningful progress in this area.

Finally, we are left with the recursive question of whether we can even trust reproducible binaries without trusting where our compilers and other toolchain components originate from. To address this, the parallel Bootstrappable Builds (bootstrappable.org) project seeks to minimize the

ABOUT THE AUTHORS

CHRIS LAMB is a freelance programmer with more than 15 years of experience developing open source software. He has contributed to the Debian operating system since 2006, and he is currently involved in the Reproducible Builds project. He is also a director of the Open Source Initiative as well as Software in the Public Interest, Inc. Further information about him can be found at ttps://chris-lamb.co.uk/. Contact him at chris@chris-lamb.co.uk.

STEFANO ZACCHIROLI is an associate professor of computer science at Université de Paris on leave at Inria, Paris, France. He is a cofounder and current CTO of the Software Heritage project. He is a member of the steering committee of the Reproducible Builds project. Further information about him can be found at http://upsilon.cc. Contact him at zack@irif.fr.

amount of binary code required to bootstrap a minimal C compiler. At the time of this article's writing, a binary as small as 6 kB is enough to activate a chain of steps from the Tiny C Compiler (TCC)[15] to GCC, from which nearly all of the toolchains can then be obtained. Ken Thompson would likely approve while still pointing out that 6 kB is too much untrusted code.

Acknowledgments

The authors would like to thank the Reproducible Builds and the wider Debian community for their feedback on this article as well as for their invaluable work on increasing the trustworthiness of FOSS. The authors also thank Giovanni Mascellani for the insightful discussions on bootstrappable builds.

References

1. K. Thompson, "Reflections on trusting trust," *Commun. ACM*, vol. 27, no. 8, pp. 761–763, 1984. doi: 10.1145/358198.358210.
2. P. C. Rigby, B. Cleary, F. Painchaud, M.-A. D. Storey, and D. M. Germán, "Contemporary peer review in action: Lessons from open source development," *IEEE Softw.*, vol. 29, no. 6, pp. 56–61, 2012. doi: 10.1109/MS.2012.24.
3. J. Rutkowska and A. Tereshkin. "Evil maid goes after TrueCrypt." The Invisible Things Lab, 2009. https://theinvisiblethings.blogspot.com/2009/10/evil-maid-goes-after-truecrypt.html (accessed Mar. 1, 2021).
4. C. Cimpanu. "Third malware strain discovered in SolarWinds supply chain attack." ZDNet, 2021. https://www.zdnet.com/article/third-malware-strain-discovered-in-solarwinds-supply-chain-attack/ (accessed Mar. 1, 2021).
5. M. Ohm, H. Plate, A. Sykosch, and M. Meier, "Backstabber's knife collection: A review of open source software supply chain attacks," in *Proc. 17th Int. Conf. Detection Intrusions Malware, Vulnerability Assessment (DIMVA 2020)*, 2020, pp. 23–43.
6. Y. Ye, T. Li, D. Adjeroh, and S. S. Iyengar, "A survey on malware detection using data mining techniques," *ACM Comput. Surv.*, vol. 50, no. 3, pp. 1–40, 2017. doi: 10.1145/3073559.
7. E. Amoroso, "Recent progress in software security," *IEEE Softw.*, vol. 35, no. 2, pp. 11–13, 2018. doi: 10.1109/MS.2018.1661316.
8. T. Durieux, C. Le Goues, M. Hilton, and R. Abreu, "Empirical study of restarted and flaky builds on Travis CI," in *Proc. 17th Int. Conf. Mining Softw. Repositories (MSR 2020)*, 2020, pp. 254–264. doi: 10.1145/3379597.3387460.
9. Q. Luo, F. Hariri, L. Eloussi, and D. Marinov, "An empirical analysis of flaky tests," in *Proc. 22nd ACM SIGSOFT Int. Symp. Found. Softw. Eng.*, 2014, pp. 643–653. doi: 10.1145/2635868.2635920.
10. C. Lamb and X. Luo, "SOURCE_DATE_EPOCH specification," Reproducible Builds, 2017. [Online]. Available: https://reproducible-builds.org/specs/source-date-epoch/
11. M. Meyer, "Continuous integration and its tools," *IEEE Softw.*, vol. 31, no. 3, pp. 14–16, 2014. doi: 10.1109/MS.2014.58.
12. B. K. Reddy Vangoor, V. Tarasov, and E. Zadok. "To FUSE or not to FUSE: Performance of user-space file systems," in *Proc. 15th USENIX Conf. File Storage Technol. (FAST 2017)*, 2017, pp. 59–72.
13. B. Laurie, "Certificate transparency," *Commun. ACM*, vol. 57, no. 10, pp. 40–46, 2014. doi: 10.1145/2659897.
14. S. Landau, "Making sense from Snowden: What's significant in the NSA surveillance revelations," *IEEE Security Privacy*, vol. 11, no. 4, pp. 54–63, 2013. doi: 10.1109/MSP.2013.90.
15. F. Bellard. "TCC: Tiny C compiler," 2003. https://bellard.org/tcc/ (accessed Oct. 5, 2020).